Travelling Salesman Problem (TSP) – Simple Notes

What is TSP?

The **Travelling Salesman Problem (TSP)** is a problem in which a salesman has to:

- Start from one city.
- Visit every city **only once**.
- Return to the starting city.
- Find the route with the **minimum total distance (cost)**.

Example

Suppose there are **4 cities**.

Distance Matrix:

```
graph = [
   [0, 10, 15, 20],
   [10, 0, 35, 25],
   [15, 35, 0, 30],
   [20, 25, 30, 0]
]
```

Program

```
from itertools import permutations
graph = [
   [0, 10, 15, 20],
   [10, 0, 35, 25],
   [15, 35, 0, 30],
   [20, 25, 30, 0]
]
cities = [1, 2, 3]
minimum_cost = float('inf')

for path in permutations(cities):
   cost = 0
   current = 0
   for city in path:
      cost += graph[current][city]
      current = city

   cost += graph[current][0]

   if cost < minimum_cost:
      minimum_cost = cost
      best_path = path

print("Best Path:", "0 ->", " -> ".join(map(str, best_path)), "-> 0")
print("Minimum Cost =", minimum_cost)
```

Step-by-Step Explanation

Step 1: Import permutations

from itertools import permutations

Explanation

- permutations() generates **all possible orders** of the cities.

Example:

(1,2,3)

(1,3,2)

(2,1,3)

(2,3,1)

(3,1,2)

(3,2,1)

Step 2: Store the Distance Matrix

```
graph = [
   [0,10,15,20],
   [10,0,35,25],
   [15,35,0,30],
   [20,25,30,0]
]
```

Explanation

The graph stores the distance between every pair of cities.

Example: Path = (1,3,2)

Complete Route:

0 → 1 → 3 → 2 → 0

graph[0][1] = 10

graph[1][3] = 25

graph[3][2] = 30

graph[2][0] = 15

Total Cost = 10 + 25 + 30 + 15 = 80

Step 3: Cities to Visit

cities = [1,2,3]

Explanation

- City **0** is the starting city.
- Therefore, we only generate permutations of the remaining cities.

Step 4: Initialize Minimum Cost

minimum_cost = float('inf')

Explanation

- float('inf') means **infinity**.
- Initially, we assume the minimum cost is very large.
- Whenever we find a smaller cost, we update it.

Step 5: Generate Every Possible Route

for path in permutations(cities):

Explanation

This loop checks every possible route.

Example:

Path 1 = (1,2,3)

Route:

0 →

1 → 2 → 3 → 0

Path 2 = (1,3,2)

Route:

0 → 1 → 3 → 2 → 0

The computer checks **all routes** one by one.

Step 6: Initialize Cost and Starting City

cost = 0

current = 0

Explanation

- cost stores the total distance travelled.
- current stores the city where the salesman is currently standing.
- Initially, the salesman starts from City 0.

Step 7: Visit Each City

for city in path:

Suppose

path = (1,3,2)

The loop will run like this:

First Iteration

Current = 0

Next City = 1

cost += graph[0][1]

Cost = 10

Update

current = 1

Second Iteration

Current = 1

Next City = 3

cost += graph[1][3]

Cost = 10 + 25 = 35

Update

current = 3

Third Iteration

Current = 3

Next City = 2

cost += graph[3][2]

Cost = 35 + 30 = 65

Update

current = 2

Step 8: Return to Starting City

After visiting all cities, the salesman is at **City 2**.

He must return to City 0.

cost += graph[current][0]

graph[2][0] = 15

So,

Total Cost = 65 + 15 = 80

Step 9: Compare Minimum Cost

if cost < minimum_cost:

Explanation

If the current route is shorter than the previous best route:

minimum_cost = cost

best_path = path

Store the new minimum cost and route.

Step 10: Display Result

print("Best Path:", "0 ->", " -> ".join(map(str, best_path)), "-> 0")

print("Minimum Cost =", minimum_cost)

Output

Best Path:

0 → 1 → 3 → 2 → 0

Minimum Cost = 80

Dry Run

Route	**Cost**
0 → 1 → 2 → 3 → 0	95
0 → 1 → 3 → 2 → 0	**80** ✅
0 → 2 → 1 → 3 → 0	95
0 → 2 → 3 → 1 → 0	80
0 → 3 → 1 → 2 → 0	95
0 → 3 → 2 → 1 → 0	95

The program chooses the route with the **minimum cost (80)**.

Algorithm

1. Start from City 0.
2. Generate all possible routes using permutations().
3. Calculate the total cost of each route.
4. Return to the starting city.
5. Compare the cost with the minimum cost.
6. Store the best route.

7. Print the minimum cost and best path.

Time Complexity

- Number of cities = **n**
- The algorithm checks **all possible routes**.

Time Complexity = O(n!)

This is why the brute-force approach is suitable only for a small number of cities.

Viva Questions

Q1. What is the Travelling Salesman Problem?
A problem of finding the shortest route that visits every city exactly once and returns to the starting city.

Q2. Which algorithm is used in this program?
Brute Force (using all possible permutations).

Q3. Why do we use permutations()?
To generate all possible orders of visiting the cities.

Q4. Why is current = city used?
It updates the salesman's current location after moving to the next city.

Q5. Why do we use float('inf')?
To initialize the minimum cost with a very large value so it can be replaced by any smaller route cost.

Q6. What is the time complexity of this solution?
O(n!), because every possible route is checked.